TRAINING AND SCALING LANGUAGE MODELS: FROM FIRST PRINCIPLES TO EFFICIENT SERVING

Module 5: Distributed Data Parallelism

Replicas, Collectives, and Global Invariants

BRAIN team

IMT Atlantique · January 2027

IMT Atlantique
Bretagne-Pays de la Loire
École Mines-Télécom

BRAIN

Lecture map

1. Distributed foundations
2. The data-parallel contract
3. Overlap and rank-aware state
4. Correctness before scaling

Guiding question

When does training on W GPUs reproduce one larger-batch update, and where can that equivalence fail?

What should be clear by the end?

1. Explain ranks, process groups, collective communication and its costs.
2. Derive DDP gradient averaging and connect it to accumulation.
3. Keep accumulation, sampling, logging, and checkpointing rank-aware.
4. Measure both numerical equivalence and scaling efficiency.

PART 1

Distributed foundations

1. Distributed foundations
2. The data-parallel contract
3. Overlap and rank-aware state
4. Correctness before scaling

Why go beyond one GPU?

Throughput

Divide work across devices to process a fixed workload sooner, or increase the workload per update.

Capacity

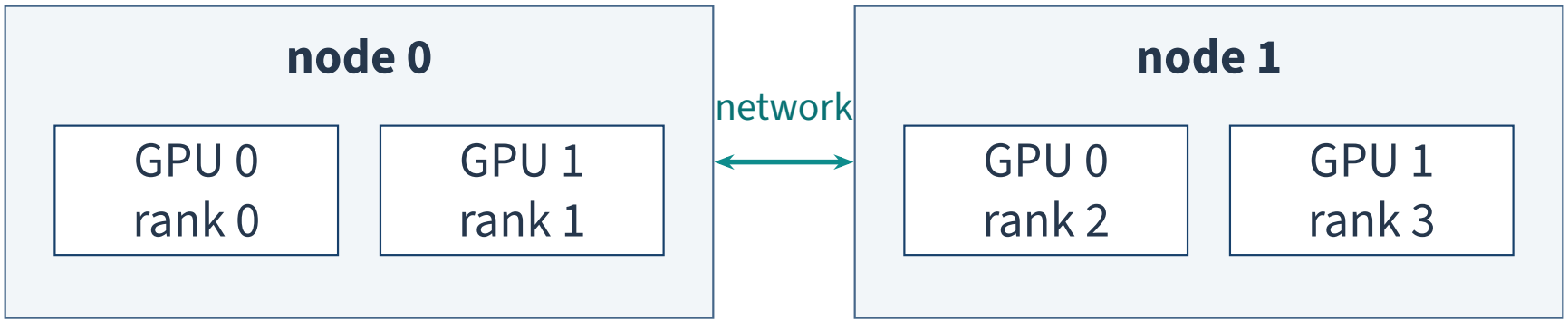
Distribute model state or computation when the model and its training state do not fit on one device.

The new cost

GPUs must exchange information. Network time, synchronization and changing the global batch can limit or change the result.

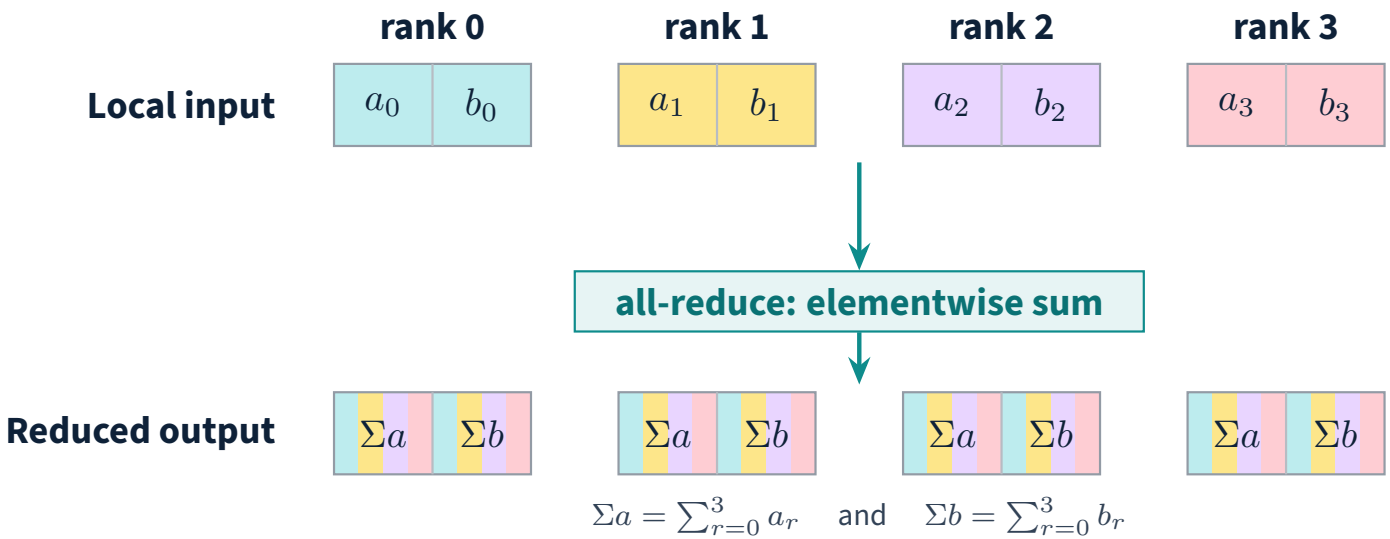
DDP first: replicate the model and split the data. FSDP next: shard model state.

A distributed job: nodes, ranks and process groups



One process per GPU in standard DDP. **Global rank** identifies a process; **local rank** identifies its GPU on the node. A **process group** defines who communicates. Here the world size is 4.

All-reduce: sum corresponding entries on every rank



DDP: divide the sum by the world size to obtain the mean gradient.

Source: NVIDIA NCCL User Guide, Collective Operations

Communication has a cost

Startup and bandwidth

Many small messages pay repeated startup costs. Large messages also depend on effective link bandwidth and topology.

Overlap

Communication can run while independent GPU work continues. Only the unhidden portion adds directly to the critical path.

Ask of every distributed design

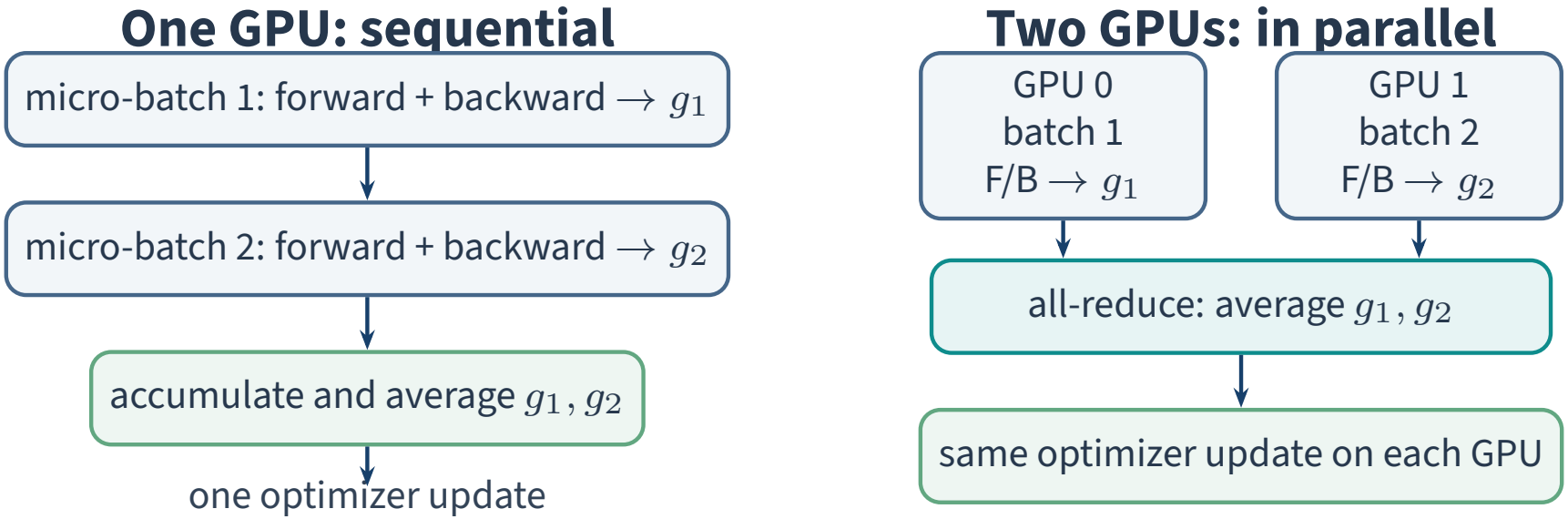
What is sent, between which ranks, how often, and what computation can overlap it?

PART 2

The data-parallel contract

1. Distributed foundations
2. The data-parallel contract
3. Overlap and rank-aware state
4. Correctness before scaling

From gradient accumulation to data parallelism



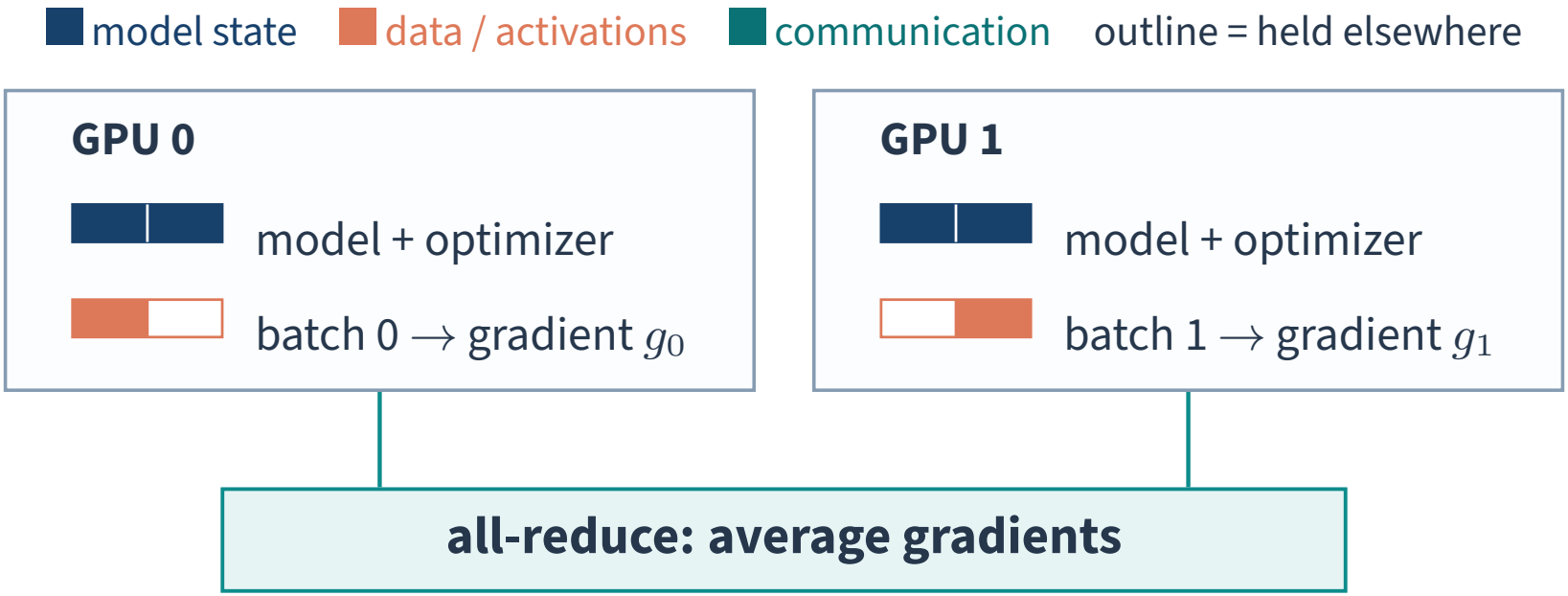
Same global mean gradient

$$\bar{g} = \frac{g_1 + g_2}{2}$$ (equal-sized micro-batches, same starting weights)

F/B = forward + backward. DDP communicates gradients and keeps model/optimizer replicas.

Randomness or batch-dependent operations can affect equivalence.

Parallelism map: DDP splits the batch



What changes? Different data on each device; the model and optimizer are replicated. After synchronization, every replica applies the same update.

Equal-size batches; identical initial model and optimizer state.

Each rank owns a full model and different examples

- one process controls one GPU
- parameters begin identical on all ranks
- local batches are disjoint parts of a global batch
- gradients synchronize before every optimizer update

Global batch

$$B_{\text{global}} = W\, B_{\text{local}}\, K$$

K is the number of accumulated micro-steps.

Gradient averaging recovers the global mean

Rank r computes a mean gradient over n_r valid tokens:

$$g_r = \frac{1}{n_r} \sum_{i \in \mathcal{B}_r} \nabla_\theta \ell_i$$

If every $n_r = n$, DDP's average is:

$$\bar{g} = \frac{1}{W} \sum_{r=0}^{W-1} g_r = \frac{1}{Wn} \sum_{r,i} \nabla_\theta \ell_i$$

Unequal valid-token counts

Averaging rank means is not the global token mean. Aggregate summed loss and counts, or weight gradients correctly.

PART 3

Overlap and rank-aware state

1. Distributed foundations
2. The data-parallel contract
3. Overlap and rank-aware state
4. Correctness before scaling

Accumulation should synchronize once per update

- Early micro-steps accumulate locally under `no_sync()`.
- The final micro-step triggers the bucket all-reduces.
- Loss normalization still targets the global mean.

Common waste

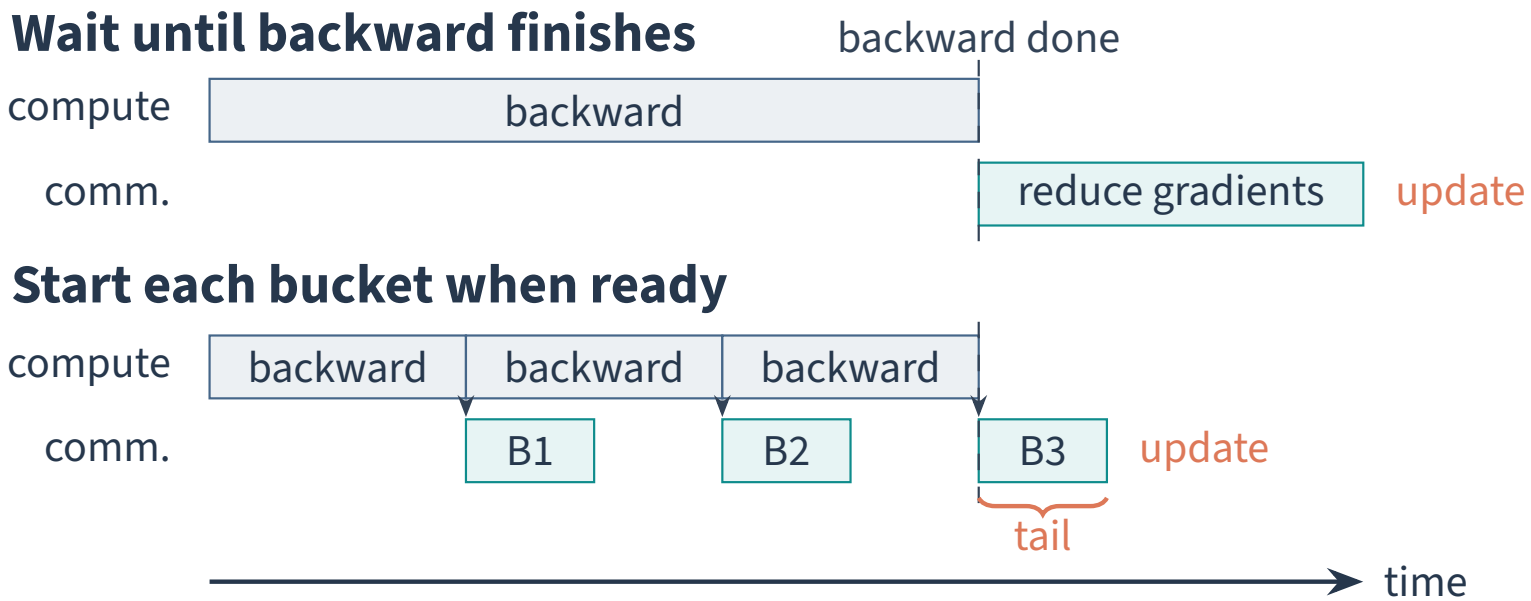
Synchronizing every micro-step communicates K times for one optimizer update.

Common bug

Never synchronizing lets replicas drift immediately after the update.

Buckets can overlap communication with backward

DDP launches an all-reduce when a bucket's gradients are ready; other layers can continue backward.



B1–B3 are gradient buckets. Only the final reduction remains after backward; the update waits for it. Timings are illustrative.

Distributed sampling is part of correctness

- Each rank needs a disjoint subset within an epoch or sampling interval.
- All ranks must agree on the shuffle seed and epoch counter.
- Dropping or padding the tail changes sample multiplicity.
- Resume must restore the sampler’s logical position.

Fast audit

Log sample IDs per rank for a few updates. Check within-step overlap, missing examples, and resume continuity.

Only some side effects should happen on every rank

Action	Every rank?	Reason
forward and backward	yes	local data contribution
optimizer update	yes	replicas remain identical
metric numerator/count	yes	later reduction
console and tracker logging	usually rank 0	avoid duplicates
full checkpoint publication	coordinated	avoid corruption
validation sampling	sharded or rank 0	declare aggregation

Checkpoint: batch accounting

Think, then compare

With $W = 8$, micro-batch 2, context 2048, and four accumulated micro-steps, compute tokens per update. How many gradient synchronizations should occur?

PART 4

Correctness before scaling

1. Distributed foundations
2. The data-parallel contract
3. Overlap and rank-aware state
4. Correctness before scaling

A five-rung correctness ladder

1. parameters are identical immediately after initialization
2. one global-batch update matches a single-process reference
3. parameters remain synchronized after the update
4. sample IDs show the intended global data coverage
5. save and resume preserve model, optimizer, RNG, and sampler state

Tolerance is part of the test

Reduction order changes floating-point rounding. Define acceptable numerical error rather than demanding bitwise equality by accident.

Scaling efficiency separates speed from more work

$$\text{speedup}(W) = \frac{t_1}{t_W}, \qquad \text{efficiency}(W) = \frac{\text{speedup}(W)}{W}$$

Strong scaling

Hold the global batch and work fixed. Ask how much faster the same update becomes.

Weak scaling

Hold work per GPU fixed. Global batch grows with W.

A trace explains poor scaling

- long exposed collectives suggest insufficient compute overlap or slow links
- rank skew suggests data loading, imbalance, or stragglers
- many small collectives suggest poor bucketing
- gaps before collectives suggest autograd readiness or synchronization barriers
- identical time with more GPUs may mean the local batch became too small

Checkpoint: equivalence failure

Think, then compare

A two-GPU run has the expected throughput, but its first update differs sharply from the single-process reference. Give a check order covering initialization, data, loss normalization, reduction, and optimizer state.

Key takeaways

1. DDP is equivalent to a global mean update only when data and normalization invariants hold.
2. Collective communication has startup and bandwidth costs; buckets can overlap it with backward.
3. Sampling, metrics, side effects, and resume logic must all understand rank.
4. Correctness comparisons come before speedup and efficiency measurements.